arXiv:2602.01322v1 [cs.LG] 1 Feb 2026

PolySAE: Modeling Feature Interactions in Sparse Autoencoders via Polynomial Decoding

Panagiotis Koromilas [1 2] **Andreas D. Demou** [1] **James Oldfield** [3] **Yannis Panagakis** [2 4] **Mihalis A. Nicolaou** [5 1]

Abstract

Sparse autoencoders (SAEs) have emerged as a promising method for interpreting neural network representations by decomposing activations into sparse combinations of dictionary atoms. However, SAEs assume that features combine additively through linear reconstruction, an assumption that cannot capture compositional structure: linear models cannot distinguish whether "Starbucks" arises from the composition of "star" and "coffee" features or merely their co-occurrence. This forces SAEs to allocate monolithic features for compound concepts rather than decomposing them into interpretable constituents. We introduce PolySAE, which extends the SAE decoder with higher-order terms to model feature interactions while preserving the linear encoder essential for interpretability. Through low-rank tensor factorization on a shared projection subspace, PolySAE captures pairwise and triple feature interactions with small parameter overhead (3% on GPT2). Across four language models and three SAE variants, PolySAE achieves an average improvement of approximately 8% in probing F1 while maintaining comparable reconstruction error, and produces 2–10× larger Wasserstein distances between class-conditional feature distributions. Critically, learned interaction weights exhibit negligible correlation with co-occurrence frequency ($r = 0.06$ vs. $r = 0.82$ for SAE feature covariance), suggesting that polynomial terms capture compositional structure, such as morphological binding and phrasal composition, largely independent of surface statistics.

[1]The Cyprus Institute [2]University of Athens [3]University of Oxford [4]Archimedes AI/Athena Research Center [5]University of Cyprus. Correspondence to: Panagiotis Koromilas <pakoromilas@di.uoa.gr>.

Preprint. February 3, 2026.

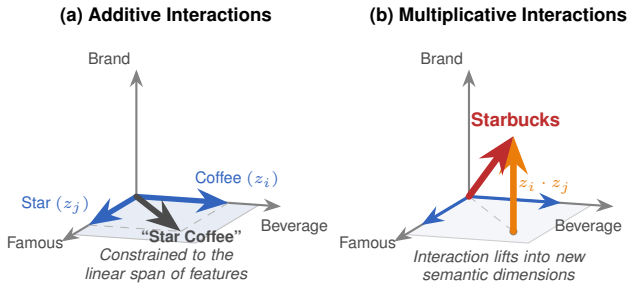

Figure 1. **Semantic Dimension Expansion via Feature Interaction.** Consider two semantic directions—*Famous* and *Beverage*—and their associated learned features *Star* and *Coffee*. **(a)** Additive interactions yield co-occurrence semantics that remain in the original feature span. **(b)** Multiplicative interactions enable representations to escape this subspace via $z_i \cdot z_j$, lifting into orthogonal dimensions (*Brand*) to capture emergent concepts like *Starbucks*."Starbucks" example from Table 3.

1. Introduction

As AI systems are increasingly deployed in real-world domains, ensuring their safety and reliability has become a critical challenge (Amodei et al., 2016; Hendrycks et al., 2021; Bengio et al., 2025). Developing interpretable models offers a promising path towards aligning AI with human values: understanding why a model produces a given output enables us to (i) monitor its reasoning (Lindsey et al., 2025), (ii) debug failure modes (Wong et al., 2021), and (iii) steer away from unwanted behavior (Rimsky et al., 2024). Mechanistic interpretability pursues this agenda at the level of neural network internals (Bereska & Gavves, 2024), aiming to uncover interpretable features and circuits within a model and thereby provide principled insights into its behavior.

Sparse Autoencoders (SAEs), grounded in the principles of sparse dictionary learning, have emerged as a leading tool for mechanistic interpretability. SAEs decompose neural network activations to recover human-interpretable features that models typically represent in superposition—encoded in overlapping directions due to limited representational capacity (Elhage et al., 2022). This framework has been shown to uncover safety-relevant concepts such as deception, bias, and harmful content, enabling targeted interventions that predictably steer model behavior (Templeton et al., 2024).

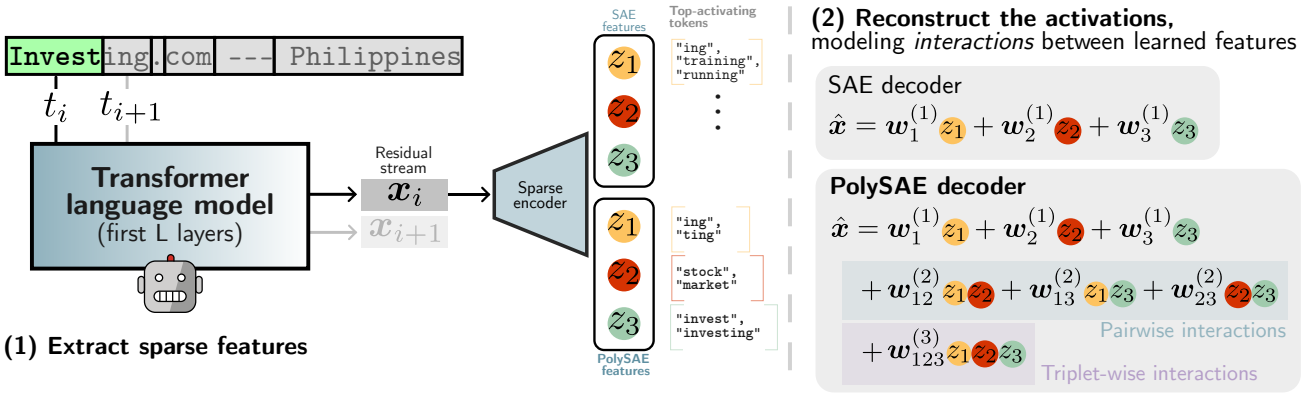

Figure 2. **An overview of PolySAE**: (1) sparse latent features are first extracted with a standard SAE encoder. (2) Activations in the residual stream are then reconstructed by modeling 2nd- and 3rd-order interactions in addition to the standard linear component. The example "**Invest**ing.com — Philippines stocks were higher after" comes from Table 4.

However, recent work has highlighted fundamental limitations of the SAE paradigm due to their reliance on the "strong" linear representation hypothesis (Engels et al., 2025; Csordás et al., 2024). Standard SAEs reconstruct activations as weighted sums of independent features, expressing each activation as a linear combination where features contribute additively. This linearity assumption raises a fundamental question: *what level of abstraction do learned features naturally capture?* The answer has direct implications for mechanistic interpretability. If features truly combine linearly, we would expect individual dictionary atoms to represent atomic components, such as morphemes, simple concepts, or basic semantic primitives, that combine through superposition to form complex expressions. Such atomic features would enable transparent circuit analysis and precise interventions on elemental building blocks of meaning.

Yet linguistic theory demonstrates that composition operates non-linearly across multiple levels of language structure. Morphologically, "administrators" is not simply the sum of stem and suffix; the combination produces a distinct lexical item with specific syntactic and semantic properties (Haspelmath & Sims, 2013). Semantically, phrasal meanings such as "kick the bucket' or proper names like "Starbucks" (Figure 1) *exhibit emergent properties irreducible to their parts* (Partee, 1995). Vanilla SAEs demonstrably succeed at many interpretability tasks, yet their linear reconstruction mechanism cannot, in principle, represent non-linear composition. Without explicit interaction mechanisms, SAEs cannot simultaneously represent atomic features and their non-linear compositions. When "Starbucks" appears in context, a linear model must either (i) allocate a dedicated feature for this compositional entity, sacrificing atomicity, or (ii) represent it through separate "star" and "coffee" features that cannot distinguish this specific composition from mere co-occurrence.

Ideally, SAEs with sufficient capacity can learn features at multiple levels of abstraction simultaneously (such as morphemes, words, phrases, and compositional expressions) co-existing as independent atoms in an overcomplete dictionary. While this leads to good reconstruction and intervention, it fundamentally limits our understanding: we cannot decompose "Starbucks" into its constituents, cannot trace how "administrators" emerge from stem and suffix binding, and cannot distinguish compositional phrases from accidental co-occurrence. The conflation of atomic and compositional features *obscures the mechanisms by which networks build complex representations from simpler parts*.

This problem connects to a *longstanding debate* in cognitive science about systematic compositionality in neural representations (Fodor & Pylyshyn, 1988). Smolensky (1990) proposed tensor product variable binding as a solution: *features bind through multilinear interactions* rather than linear superposition, allowing networks to maintain atomic constituents while representing their combinations. In this framework, "administrators" would be represented not as a single indivisible feature, but as an explicit *composition* of stem and suffix, where the tensor product captures the binding operation. For interpretability of modern LLMs, this principle is critical: to understand how networks compose meaning, our tools must themselves model compositional structure faithfully. However, explicit tensor products are computationally prohibitive for overcomplete sparse codes with tens of thousands of features, requiring methods that capture multilinear interactions while remaining tractable.

In this work, we introduce the **Polynomial Sparse Autoencoder (PolySAE)** (Figure 2), a sparse autoencoder that extends vanilla SAEs with explicit feature interaction terms. PolySAE preserves a linear encoder for interpretability while extending the decoder with quadratic and cubic terms that model pairwise and triple feature interactions. Through low-rank tensor factorization on a shared projection subspace, PolySAE captures compositional structure in

a tractable manner, adding a small parameter overhead (3% for GPT2 small). Critically, PolySAE is a strict *generalization of standard SAEs* that enables capturing multiplicative (non-additive) concept interactions. Setting interaction coefficients to zero recovers vanilla SAE behavior, allowing PolySAE to be readily applied to existing SAE variants, including TopK (Gao et al., 2025), BatchTopK (Bussmann et al., 2024), and Matryoshka (Bussmann et al., 2025b).

We summarize below our **four main contributions**:

C1. We introduce **PolySAE**, a sparse autoencoder with a polynomial decoder that explicitly **models quadratic and cubic feature interactions** while preserving a linear encoder for interpretability. Through low-rank tensor factorization, PolySAE adds small parameter overhead (3% for GPT2 small) and can be **readily applied** to existing SAE variants (TopK, BatchTopK, Matryoshka).

C2. Across **four language models** of different scales (GPT-2 Small, Pythia-410M/1.4B, Gemma-2-2B) and **three sparsification strategies**, PolySAE achieves an **average 8% F1 improvement**, while maintaining comparable reconstruction error.

C3. PolySAE produces **2–10× larger Wasserstein distances** between class-conditional feature distributions, indicating more separated semantic structure in the learned representations.

C4. We show that learned interaction weights exhibit **negligible correlation with co-occurrence frequency** (r = 0.06 vs. r = 0.82 for SAE feature covariance), and provide qualitative examples demonstrating that **polynomial terms capture compositional structure** such as morphological binding, phrasal composition, and contextual disambiguation.

2. Related Work

Sparse dictionary learning. In sparse dictionary learning, signals are represented as sparse linear combinations of overcomplete basis elements (Mallat & Zhang, 1993), an approach also integrated into neural network architectures (Hinton & Salakhutdinov, 2006; Lee et al., 2007; Konda et al., 2014). Sparse Autoencoders (SAEs) recently emerged as a leading paradigm for feature discovery in large language models (Huben et al., 2024; Bricken et al., 2023), scaling to millions of features (Gao et al., 2025). Subsequent work has produced architectural variants including BatchTopK (Bussmann et al., 2024), Matryoshka (Bussmann et al., 2025a), Gated (Rajamanoharan et al., 2024a), and JumpReLU (Rajamanoharan et al., 2024b) SAEs, with standardized benchmarks enabling systematic comparison (Karvonen et al., 2025). However, all these methods assume features combine additively through linear reconstruction.

Modeling feature interactions. Multiplicative interactions between features have a rich history in deep learning (Jayakumar et al., 2020), from early bilinear models for visual data (Tenenbaum & Freeman, 1996; Freeman & Tenenbaum, 1997) to modern gating mechanisms (Shazeer, 2020). Feature interactions through the Hadamard product (Chrysos et al., 2025) serve as a powerful conditioning mechanism (Perez et al., 2018; Dumoulin et al., 2017), while multiplicative structure also enables parameter-efficient mixture-of-experts (Oldfield et al., 2025a). Recent work has explored multiplicative interactions for interpretability: Bilinear MLPs (Pearce et al., 2025) model pairwise feature interactions enabling weight-based interpretability, while Gauderis & Dooms (2025) propose fully interpretable architectures based on tensor networks. We extend this line of work by modeling feature interactions in the SAE setting.

Polynomials. One natural way to model higher-order interactions is through polynomials (Shin & Ghosh, 1991). In deep learning, polynomials have been used for a variety of applications, such as image generation (Chrysos et al., 2020; 2021), classification (Babiloni et al., 2021; Chrysos et al., 2022a), privacy preservation (Zhang et al., 2019), interpretability (Dubey et al., 2022), and dynamic safety guardrails (Oldfield et al., 2025b). The work most closely related to ours is the Bilinear Autoencoder (BAE) (Dooms & Gauderis, 2025), which similarly introduces interaction terms for interpretability. The key difference lies in the level at which interactions are modeled: BAE captures pairwise interactions between input neurons, whereas PolySAE models interactions directly between learned sparse features, including higher-order terms. As a result, PolySAE preserves the interpretability of linear SAE latents while explicitly allocating capacity to non-additive feature composition.

3. Sparse Polynomial Decoding

3.1. Preliminaries

Notation. Bold lowercase letters denote vectors and bold uppercase letters denote matrices. The i-th column of $\boldsymbol{M}$ is $\boldsymbol{m}_i$, and $\boldsymbol{M}_{:,1:r}$ denotes its first r columns. We use $*$ for the Hadamard product, $\otimes$ for the Kronecker product, and $\odot$ for the Khatri–Rao product. $\mathbb{R}^d$ and $\mathbb{R}^{d_{\text{sae}}}$ denote the activation and sparse-code spaces, with $d_{\text{sae}} \gg d$. $\mathcal{S}(\cdot)$ denotes a sparsification operator, such as Top-K or BatchTop-K.

Sparse Autoencoders. Sparse autoencoders (SAEs) build on overcomplete dictionary learning (Olshausen & Field, 1997) to decompose neural activations into a sparse set of latent features. Given activations $\boldsymbol{x} \in \mathbb{R}^d$ from an intermediate layer of a pretrained network, an SAE learns a sparse code $\boldsymbol{z} \in \mathbb{R}^{d_{\text{sae}}}$ with $d_{\text{sae}} \gg d$ and reconstructs via $\hat{\boldsymbol{x}} = \boldsymbol{b}_{\text{dec}} + \boldsymbol{D}\boldsymbol{z}, \boldsymbol{z} = \mathcal{S}\big(\text{ReLU}(\boldsymbol{E}^\top \boldsymbol{x} + \boldsymbol{b}_{\text{enc}})\big)$ where $\boldsymbol{E}$ is a linear encoder, $\boldsymbol{D}$ is the decoder (dictionary), and $\mathcal{S}$ enforces sparsity. The overcomplete latent space allows